

Standalone (Air-Gapped) MITM Proxy Service

Overview

The **Standalone MITM Proxy Service** is a special deployment mode of our "man-in-the-middle" proxy application where all its microservice components run in a single process or container. This mode allows the entire system to function offline (even in air-gapped environments) by embedding each service in-memory. In essence, it combines what is normally a multi-service architecture into **one executable**, simplifying deployment and enabling end-to-end testing without external dependencies.

Motivation for a Standalone Mode

Originally, the MITM Proxy platform is composed of multiple microservices (caching, HTML processing, graph generation, text transformation, etc.), each deployed separately. While this microservice architecture offers scalability, it complicates integration testing and deployments in isolated environments. Key motivations for a standalone, all-in-one mode include:

- **Integration Testing:** Enable fast, **end-to-end tests** that spin up the entire system in-memory, avoiding flaky network calls between services. This provides high confidence that workflows function correctly across service boundaries.
- **Air-Gapped Deployments:** Allow running the proxy in isolated or offline environments (no external network access) by packaging all components together. All inter-service communication occurs internally, so the system can run on a single machine without internet.
- **Simplified Prototyping and Demos:** For quick prototypes, lightweight real-world scenarios or MVP deployments, a single executable is easier to deploy (just Python and required packages) compared to managing multiple services. This "all-in-one" mode provides a convenient way to showcase functionality with minimal setup.

In short, the standalone version improves testability and accessibility of the MITM Proxy by removing external service dependencies when needed.

Architecture of the Single-Process Deployment

In the standalone configuration, **all normally separate services are started within one process**. They can either be instantiated as sub-applications or simply initialized in test mode. The architecture ensures that inter-service calls are routed in-memory rather than over HTTP sockets:

- All services (the core MITM proxy API plus dependencies like the caching service, HTML/HTML graph services, text comprehension service, etc.) are initialized together in one FastAPI application or within one Python process.
- Each service's API endpoints are available, but instead of running on separate network ports, they are invoked internally. The services share the same runtime and memory space (one container/process) for this mode.
- A special **client abstraction layer** is used for service-to-service communication. This layer can operate in two modes:

- **Live HTTP Requests:** In a normal deployment, services call each other via external HTTP requests (e.g., using the `requests` or `httpx` library to call another service's URL).
- **In-Memory Calls (Test Mode):** In the standalone mode, the client layer is configured to use FastAPI's TestClient or HTTPX's ASGI client to call the target service's app directly in memory, bypassing network sockets. The TestClient from FastAPI/Starlette provides the same interface as an HTTP client ¹, so existing request code works the same, but the calls never leave the process.

By switching the client abstraction to "test mode," every internal API call is handled by an in-memory FastAPI app, which **eliminates network overhead and external dependencies**. This architecture pattern essentially creates a miniature version of the entire platform that runs as a single unit.

In-Memory Services and Test Clients

A critical implementation detail is how we achieve internal calls between these combined services. We leverage FastAPI's testing tools to simulate HTTP interactions locally:

- **FastAPI TestClient:** FastAPI (via Starlette) provides a `TestClient` class that allows calling API endpoints as if via HTTP, but actually running the request entirely in memory ². It is built on HTTPX and models the Requests library API, meaning our service calls (GET/POST, etc.) use the same interface whether they go to a live server or an in-memory one.
- **Service Client Abstraction:** Each microservice in our architecture has a client module that normally issues HTTP calls to other service endpoints. We introduced a configuration in these clients to choose between real HTTP calls and TestClient calls. In standalone mode, this configuration is set to use TestClient for all inter-service communication.
- **Dependency Injection:** In some cases, we can override dependencies to provide in-memory clients. For example, FastAPI allows overriding dependencies during testing. We could replace a dependency that provides an HTTP client with one that uses an internal app client. This way, when service A tries to call service B, it actually calls the local instance of service B's app directly. By doing so, we **avoid actual network calls** between services ³, which not only makes tests faster but also enables the whole system to run without network access.

Using these techniques, when the MITM Proxy service (front-end) calls, say, the caching service or the HTML rendering service, the call is fulfilled by invoking the function in-memory and returning the result, as opposed to making an external HTTP request. This is completely transparent to the service logic – the code "thinks" it's making a normal request, but the TestClient catches it and routes it internally.

Setting Up the Standalone Test Environment

For running integration tests or an offline instance using this standalone approach, the setup would look like the following:

1. **Initialize All Services:** The test harness or startup script will import and create instances of each needed FastAPI service (e.g., instantiate the caching service app, the HTML/Graph service app, etc., as Python objects). Each service is configured in a special **test mode** where it uses in-memory data stores (e.g., in-memory cache instead of a cloud cache, memory or temp files instead of cloud storage) as much as possible.
2. **Configure Clients for In-Memory Routing:** Before starting the main front-end service (the MITM proxy API), configure its service clients to use the in-memory TestClient. This may involve setting a flag or environment variable that our request abstraction layer checks. At this point, each service knows how to reach the others internally.

3. **Launch the Combined Application:** Start the main FastAPI application (the MITM proxy service). As part of its startup, it can also spawn the other service applications if not already started. Since everything is running in one process, this might simply mean ensuring the other apps are imported and ready. No separate Uvicorn processes are needed for the dependencies in test mode.
4. **Run Tests or Interact:** With everything in one process, we can run our integration tests. For example, a test can simulate traffic by calling the MITM proxy API (using a TestClient on the main app) which in turn triggers internal calls to the subordinate services. We verify the end-to-end behavior (e.g., that a request through the proxy hits the caching service, the HTML parser, etc., and returns the expected result).
5. **Tear Down:** After tests, shut down the apps and clear any in-memory state. However, because each service is lightweight and initializes in a few milliseconds, they can also be re-initialized for each test case without significant overhead (on the order of tens of milliseconds per service initialization, which is negligible for unit/integration tests).

This approach essentially boots up a **miniature version of our production environment** inside the test suite. The benefit is we are exercising the real code paths across services, not mocking them out, while still running quickly and without external infrastructure.

Deployment as a Single Executable

Beyond testing, the standalone MITM proxy can serve as a **single-executable deployment** of the entire system:

- **Single Container or Process:** Instead of deploying five or six separate services (caching, HTML graph, text, etc.) and managing their networking, we package the code into one container image or Python package. The architecture (as noted) runs all components in one Python process. This greatly simplifies deployment in environments where running multiple services is impractical (for example, a small on-prem server or an edge device that may be air-gapped).
- **Serverless and Function Compatibility:** Because our services were designed for serverless deployment as separate functions, combining them doesn't break functionality. We maintain the FastAPI apps for each service, but now we have the option to deploy them together. It's even conceivable to run this unified app on a serverless platform as a single function (though typically it would run in a container or VM for an air-gapped scenario).
- **Resource Sharing:** All services in one process can share resources more easily (like an in-memory cache). However, they also share the same process resources, so heavy workloads might still be better on separate services. The standalone mode is intended for light usage scenarios and testing rather than full production load.
- **Cloud Dependencies:** In the initial version of this standalone deployment, we assume that any external cloud resources (for example, if the text comprehension service normally calls a cloud API or if storage would normally go to cloud storage) are either not needed or are accessible in the environment. We have not fully stubbed out or simulated external cloud services in-memory for the first iteration. Instead, the expectation is that those services are either **within the same environment** or can be reached if needed (e.g., if running in a company's cloud account or with appropriate offline access). In many cases, though, the design tries to eliminate the need for external calls by using the in-memory cache and local processing.

In practice, the standalone application could be started with a simple command (after installing the combined package via PyPI) and it would spin up all the components internally. This makes it very appealing for quick deployments where installing a bunch of separate services would be overkill.

Packaging and CI/CD

To distribute the standalone proxy easily, we leverage Python packaging:

- The code for this unified mode is included in the project and is also **packaged as a PyPI package**. We already have continuous integration (CI) via GitHub Actions to publish packages, so the standalone version will be released through the same pipeline. All the constituent services' code is bundled into this package.
- By installing the package (e.g., via `pip install mitmproxy-standalone`), a user gets all necessary components. The package may provide a console script entry-point to launch the unified server.
- Because everything runs in-process, the only requirements to run the standalone service are Python itself and the required Python libraries. No external system services or containers are needed for the dependencies, which aligns with the goal of simplicity.

This packaging approach ensures that even a completely isolated environment (with no internet access at runtime) can run the MITM proxy service as long as the package (and any of its library dependencies) have been installed. For truly air-gapped sites, one could pre-download the wheels or use an internal PyPI mirror to install the package.

Use Cases and Limitations

Primary Use Case – Integration Testing: The initial focus of this mode is to facilitate comprehensive integration tests. Developers can write tests that exercise the **full request flow** through the MITM proxy and its helpers. Because each test can bring up a full in-memory stack in a few hundred milliseconds at most, this fits within typical test suite execution times. We can now validate complex workflows (involving caching, HTML parsing, graph building, etc.) deterministically and quickly.

Secondary Use Case – Lightweight Deployments: The standalone service can handle light, real-world usage. For example: - Deploying the proxy in a **development or staging environment** quickly, without replicating the entire microservice infrastructure. - Running a demo for a client entirely on a laptop (no network required aside from the local loopback). - An **MVP deployment** for a small-scale pilot project where setting up all the microservices in the cloud is not justified.

It's important to note that while this unified deployment is convenient, it may not be suitable for heavy production traffic. All components share one process and machine, so it won't horizontally scale like true microservices. Nonetheless, it can handle moderate workloads and is ideal for functional testing and isolated runs.

Limitations: In the first version of the standalone mode, some external integrations might still be assumed to exist. For instance, if the system normally relies on a cloud-based database or an external API (for things like advanced text analysis), the standalone mode currently assumes those are available *within the same environment or network* (i.e., not fully mocked in-memory). Future enhancements could further encapsulate or simulate these externals if complete offline operation is required for those features. Additionally, running everything in one process means any memory leaks or heavy CPU tasks in one component could affect others – something to consider for long-running use.

Conclusion

The standalone (air-gapped) version of the MITM proxy service transforms our multi-service architecture into a self-contained application. By using in-memory service instances and FastAPI's TestClient mechanism, we achieve a powerful testing framework and a simplified deployment option. This approach demonstrates the flexibility of our design: the same codebase can be run as independent serverless functions, as microservices on Kubernetes, or as a one-shot monolithic application, depending on the need. Going forward, this will greatly enhance our ability to **test workflows end-to-end** with minimal overhead and deploy the proxy in environments that were previously challenging (or impossible) to support. The result is a more robust, testable, and portable MITM proxy platform that can run virtually anywhere.

1 2 Testing - FastAPI

<https://fastapi.tiangolo.com/tutorial/testing/>

3 Contract Testing with HTTPX - Part 2 - The FastAPI Expert

<https://fastapiexpert.com/blog/2023/03/14/contract-testing-with-httpx---part-2/>